Reducing the Cost of Muon with Monarch-Parameterized Layers

Ruslan Kabirov
MIPT
Moscow, Russia
kabirov.ra@phystech.edu

Alexandr Shestakov
MIPT
Moscow, Russia
ashestakov@phystech.edu

Abstract

Muon has recently gained attention as an optimizer for neural network training because of its use of orthogonalized matrix-valued updates [1, 2]. In its standard form, Muon relies on Newton–Schulz iterations to approximate the polar factor of a matrix update, which introduces additional computational overhead. We study whether this cost can be reduced by replacing selected dense layers with Monarch-parameterized layers [3]. This structure allows Newton–Schulz iterations to be applied to smaller block matrices and creates opportunities for parallel block-wise computation. We evaluate the proposed approach in pretraining experiments with a compact GPT-2 style model and compare it against Muon and AdamW baselines. We also evaluate a Dion2-inspired random column-partitioned variant as an ablation and find that it does not noticeably improve iteration-wise convergence over the simpler block-wise Monarch Muon update. Our goal is to reduce the cost of Muon's orthogonalization step while keeping validation loss and perplexity close to the baseline methods.

Keywords: Muon optimizer, Monarch matrices, matrix orthogonalization, Newton–Schulz iteration, polar decomposition, structured matrices, computational efficiency.

1 Introduction

1.1 Muon and orthogonalized updates

Muon was introduced as an optimizer for hidden layers in neural networks and has recently demonstrated strong empirical performance in deep learning [1]. More recent work has shown that Muon can scale to large language model training when combined with appropriate update scaling and weight decay choices [2]. Unlike standard element-wise adaptive optimizers, Muon uses matrix-valued updates and explicitly controls their geometry through an orthogonalization step. A central operation in Muon is the computation of the orthogonal factor $UV^\top$, where

$$W = U\Sigma V^\top$$

denotes the singular value decomposition of a matrix W. In the context of Muon, W denotes the matrix-valued update to which the orthogonalization step is applied. The factor $UV^\top$ can be interpreted as the orthogonal component of W, and it plays a key role in shaping the direction of the optimizer update [1, 2].

Computing this factor exactly by singular value decomposition is too expensive to perform repeatedly during training. Therefore, practical implementations of Muon usually approximate the orthogonalization step using Newton–Schulz iterations [1]. Newton–Schulz iteration is an iterative matrix procedure that approximates the orthogonal factor without explicitly computing a full SVD. Although

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

this approximation is substantially cheaper than exact decomposition, it is still applied at every optimization step. For large dense matrices, repeated orthogonalization may account for a nontrivial part of the total optimizer cost. As model sizes continue to grow, this operation can become a practical bottleneck that limits the scalability of Muon [2].

1.2 Directions for improving Muon efficiency

Recent work suggests two broad directions for improving the efficiency of Muon-type optimizers. The first direction is to modify the optimization algorithm itself or the orthogonalization procedure used inside it. Scion studies training with norm-constrained linear minimization oracles and introduces normalization mechanisms that are related to the stability of orthogonalized updates [4]. Dion proposes distributed orthonormalized updates and replaces Newton–Schulz-style orthogonalization with a power-iteration-based approach that is more suitable for parallel and distributed settings [5]. Dion2 reduces the matrix size used inside Muon by applying orthogonalization to smaller randomly selected submatrices [6]. MuonBP proposes block-periodic orthogonalization, reducing the frequency or scope of expensive orthogonalization steps [7]. Other works focus more directly on improving Newton–Schulz or related matrix sign iterations, such as Gram Newton–Schulz [8] and Polar Express [9]. These methods primarily focus on changing the optimizer or reducing the cost of the matrix operation performed by the optimizer.

The second direction, which we focus on in this work, is to exploit structure in the matrix parameters themselves. Instead of applying Muon to fully dense layers, one can replace selected dense matrices with structured parameterizations that are cheaper to store, multiply, and orthogonalize. Monarch matrices are a natural candidate for this purpose: they represent a dense-like linear transformation using permutation and block-diagonal factors, and prior work has shown that such parameterizations can provide favorable efficiency–quality trade-offs in neural network training [3]. This suggests that structured layers may reduce not only the cost of the forward and backward passes, but also the cost of Muon's Newton–Schulz orthogonalization step.

1.3 Monarch-structured layers for Muon

In this paper, we investigate whether the overhead of Muon can be reduced by replacing dense matrix layers with structured Monarch-parameterized layers [3]. Instead of assuming that an arbitrary dense matrix can be exactly decomposed into Monarch form, we restrict selected trainable matrices to a structured class of the form

$$W = PLP^\top R,$$

where P is a fixed permutation matrix, and L and R are trainable block-diagonal matrices, each consisting of two blocks. This parameterization does not represent every possible dense matrix. Rather, it replaces a dense layer with a structured surrogate that has fewer effective degrees of freedom and enables more efficient matrix operations.

This structured parameterization is particularly relevant for Muon because the expensive orthogonalization step can be applied to smaller block-diagonal factors instead of the full dense matrix. Since the blocks are independent, Newton–Schulz iterations can be performed separately for each block. This not only reduces the effective matrix size involved in orthogonalization, but also exposes additional parallelism across blocks. Therefore, Monarch-structured layers may reduce the cost of Muon both through smaller matrix operations and through more parallel execution. This idea is complementary to optimizer-level methods such as Dion, Dion2, MuonBP, Gram Newton–Schulz, and Polar Express [5–9].

1.4 Research question and contributions

The motivation for this approach is that full dense parameterization and full-matrix orthogonalization may be unnecessarily expensive for some parts of neural networks. A structured layer can preserve useful matrix interactions through permutations and block-diagonal factors while substantially reducing computational cost [3]. However, this restriction also introduces a trade-off. Since Monarch-structured layers cannot express all dense matrices, the model may lose representational flexibility, and the optimizer update may differ from that of standard Muon applied to dense layers. The main challenge is therefore to determine whether the efficiency gains outweigh the possible loss in training quality.

The central research question of this work is whether Muon can be made substantially cheaper by using Monarch-structured layers without sacrificing training performance. More specifically, we ask which structured layer designs and update strategies provide the best trade-off between computational efficiency, parallel execution, and model quality. By studying this question, we aim to clarify whether Monarch-parameterized layers can serve as a practical mechanism for scaling Muon to larger models and more expensive training regimes.

Contributions. Our contributions can be summarized as follows:

- We propose replacing selected dense layers with Monarch-parameterized layers in order to reduce the computational cost of Muon.
- We show how the block structure of Monarch layers allows Newton–Schulz iterations to be applied to smaller independent matrices, creating opportunities for parallelization.
- We analyze the computational cost of the proposed structured Muon variant and compare it with standard Muon applied to dense layers.
- We evaluate a Dion2-inspired random column-partitioned update as an ablation to test whether additional random coupling between Monarch blocks improves convergence.
- We empirically find that the simple block-wise Monarch Muon update provides the most favorable trade-off in our setting, while the Dion2-inspired random column-partitioned variant does not noticeably improve convergence over the basic Monarch update.

Outline. The rest of the paper is organized as follows. Section 2 formalizes the problem statement and evaluation criteria. Section 3 describes the proposed structured Muon variant and the Dion2-inspired ablation. Section 4 discusses related work. The following sections present the experimental setup and compare the resulting methods against Muon and AdamW baselines.

2 Problem statement

2.1 General optimization setting

We consider the problem of training a neural network f_Θ with a collection of large matrix-valued parameters. This setting is especially relevant for large language models, where many layers contain dense matrices of substantial size, such as attention projections and feed-forward layers. Let

$$\mathcal{W} = \{W_1, \dots, W_s\}$$

denote the subset of trainable matrix parameters to which Muon-style optimization may be applied.

The model is trained by minimizing a task-dependent empirical objective

$$\min_{\Theta} \ \mathcal{L}(\Theta), \tag{1}$$

where $\mathcal{L}$ may correspond, for example, to a language modeling loss in the case of LLM pretraining. The exact form of the loss is not essential for the proposed method. The key assumption is that the model contains sufficiently large matrix parameters for which the cost of Muon's orthogonalization step becomes non-negligible.

2.2 Standard dense Muon update

For a dense matrix parameter $W \in \mathcal{W}$, standard Muon maintains a momentum matrix and applies an orthogonalized update [1, 2]:

$$G_t = \nabla_W \mathcal{L}(\Theta_{t-1}), \tag{2}$$
$$M_t = \beta M_{t-1} + G_t, \tag{3}$$
$$O_t = \mathrm{NS}(M_t), \tag{4}$$
$$W_t = W_{t-1} - \alpha O_t, \tag{5}$$

where α is the learning rate, β is the momentum coefficient, and $\mathrm{NS}(\cdot)$ denotes the Newton–Schulz approximation of the orthogonal factor.

The computational bottleneck is the repeated application of Newton–Schulz iterations to large dense matrices. Since this operation is performed during optimizer steps, the relevant efficiency metric in this work is not only the number of training iterations, but also the wall-clock time spent on optimization steps.

2.3 Block-structured parameterization

To reduce the cost of Muon's orthogonalization step, we replace selected dense matrices with structured block-parameterized matrices. In particular, we use Monarch-parameterized layers [3]. For a selected matrix W, we introduce an effective structured matrix

$$\widetilde{W} = PLP^\top R, \tag{6}$$

where P is a fixed permutation matrix, and L and R are trainable block-diagonal factors. In our main setting, each factor consists of two trainable blocks:

$$L = \mathrm{diag}(L^{(1)}, L^{(2)}), \tag{7}$$

$$R = \mathrm{diag}(R^{(1)}, R^{(2)}). \tag{8}$$

Thus, instead of optimizing the dense matrix W directly, we optimize the block parameters

$$\theta_W = \{L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}\}.$$

This replacement should be understood as a structured parameterization of the original matrix parameter. The matrix is not assumed to be exactly decomposable into Monarch form. Rather, the dense parameter is replaced by a block-structured surrogate that reduces the size of the matrices involved in Muon's orthogonalization step.

2.4 Block-wise Muon update

Instead of applying Muon to the full effective matrix $\widetilde{W}$, we apply Muon independently to the trainable blocks. For any block

$$B \in \{L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}\},$$

the update is

$$G_t^B = \nabla_B \mathcal{L}(\Theta_{t-1}), \tag{9}$$

$$M_t^B = \beta M_{t-1}^B + G_t^B, \tag{10}$$

$$O_t^B = \mathrm{NS}(M_t^B), \tag{11}$$

$$B_t = B_{t-1} - \alpha O_t^B. \tag{12}$$

After the block updates are applied, the effective matrix is reconstructed as

$$\widetilde{W}_t = PL_tP^\top R_t. \tag{13}$$

The advantage of this formulation is that Newton–Schulz iterations are applied to smaller block matrices rather than to the full dense matrix. These block-wise operations are independent and can therefore be executed in parallel.

2.5 Optimization-time objective

The central goal of this work is to reduce the wall-clock time spent on Muon optimizer steps while preserving model quality. We denote by

$$T_{\mathrm{opt}}$$

the total time spent inside optimizer updates, including momentum updates, Newton–Schulz orthogonalization, and parameter updates. The proposed method is successful if it reduces T_{opt} relative to dense Muon while keeping the final validation quality close to the baseline.

For language modeling experiments, we report validation loss and perplexity:

$$\mathrm{PPL} = \exp(\mathcal{L}_{\mathrm{val}}). \tag{14}$$

More generally, the quality metric can be replaced by the validation metric appropriate for the task. The desired trade-off is

$$\frac{|Q_{\text{method}} - Q_{\text{baseline}}|}{Q_{\text{baseline}}} \leq \varepsilon, \tag{15}$$

where Q denotes the relevant validation metric, such as perplexity for language modeling, and ε is chosen to be on the order of a few percent.

Thus, the main question is whether replacing large dense matrix parameters by Monarch block-structured parameters can reduce the optimization-step time of Muon without substantially degrading final validation performance.

3 Theory

3.1 Monarch-parameterized model

We consider a selected dense linear layer with weight matrix W. Instead of training W as a fully dense matrix, we replace it with a Monarch-parameterized matrix [3]

$$\widetilde{W} = PLP^\top R, \tag{16}$$

where P is a fixed permutation matrix, and L and R are trainable block-diagonal matrices. In our setting, each of these matrices consists of two trainable blocks:

$$L = \operatorname{diag}(L^{(1)}, L^{(2)}), \tag{17}$$

$$R = \operatorname{diag}(R^{(1)}, R^{(2)}). \tag{18}$$

Thus, the trainable parameters of one Monarch layer are

$$\theta = \{L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}\}.$$

This parameterization restricts the class of possible weight matrices. In other words, $\widetilde{W}$ is not an arbitrary dense matrix, and not every dense matrix W can be represented in this form. However, the structure preserves interactions between different coordinate groups through the permutation matrix P, while the block-diagonal factors make matrix operations cheaper and more suitable for parallel computation [3].

The main idea of our method is to apply Muon not to the full effective matrix $\widetilde{W}$, but to the smaller trainable blocks $L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}$. This changes the orthogonalization problem from one large Newton–Schulz computation to several smaller independent computations.

3.2 Newton–Schulz orthogonalization

Muon uses an approximate orthogonalized update. Given a matrix A, we define the Newton–Schulz approximation as follows [1, 2]. First, the matrix is normalized:

$$X_0 = \frac{A}{\|A\|_F}. \tag{19}$$

Then, for $k = 0, \ldots, K-1$, we apply

$$S_k = X_k X_k^\top, \tag{20}$$

$$X_{k+1} = aX_k - bS_k X_k + cS_k^2 X_k, \tag{21}$$

where in our experiments we use

$$K = 5, \qquad (a, b, c) = (3.4445,\ 4.7750,\ 2.0315).$$

The final approximation is

$$\mathrm{NS}(A) = X_K. \tag{22}$$

In standard Muon, this procedure is applied to a full momentum matrix. In the proposed method, it is applied separately to each Monarch block. Alternative improvements of the same general orthogonalization component have been studied in Gram Newton–Schulz and Polar Express [8, 9].

3.3 Block-wise Muon update

For each trainable block

$$B \in \{L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}\},$$

we maintain a separate momentum matrix M_t^B. At training step t, the gradient with respect to block B is

$$G_t^B = \nabla_B \mathcal{L}(\Theta_{t-1}), \tag{23}$$

where Θ denotes all trainable model parameters. The block-wise Muon update is then given by

$$M_t^B = \beta M_{t-1}^B + G_t^B, \tag{24}$$

$$O_t^B = \mathrm{NS}(M_t^B), \tag{25}$$

$$B_t = B_{t-1} - \alpha O_t^B, \tag{26}$$

where α is the learning rate and β is the momentum coefficient.

After all four blocks are updated, the effective layer matrix is reconstructed as

$$\widetilde{W}_t = P L_t P^\top R_t. \tag{27}$$

Thus, the optimizer never needs to apply Newton–Schulz iterations to the full dense effective matrix $\widetilde{W}_t$. Instead, it orthogonalizes four smaller matrices independently. This structure-based reduction is complementary to methods that shrink or periodically orthogonalize matrices inside the optimizer itself [6, 7].

3.4 Algorithm

The full training procedure for a Monarch-parameterized layer with block-wise Muon updates is shown in Algorithm 1.

Algorithm 1 Block-wise Muon for Monarch-parameterized layers

Require: Dataset $\mathcal{D}$, model f_Θ, learning rate α, momentum coefficient β, Newton–Schulz iterations K
Require: Monarch blocks $L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}$ constructed by the Monarch layer
$\mathcal{B} \leftarrow \{L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}\}$
Initialize $M_0^B \leftarrow 0$ for all $B \in \mathcal{B}$
for training step $t = 1, 2, \ldots$ **do**
  Sample a mini-batch from $\mathcal{D}$
  Construct $\widetilde{W}_{t-1} = P L_{t-1} P^\top R_{t-1}$
  Compute $\mathcal{L}(\Theta_{t-1})$ and gradients by backpropagation
  for each block $B \in \mathcal{B}$ **do**
    $G_t^B \leftarrow \nabla_B \mathcal{L}(\Theta_{t-1})$
    $M_t^B \leftarrow \beta M_{t-1}^B + G_t^B$
    $B_t \leftarrow B_{t-1} - \alpha\ \mathrm{NS}_K(M_t^B)$
  end for
  Update non-Monarch parameters according to the chosen experimental protocol
end for

3.5 Ablation: Dion2-inspired random column-partitioned update

In addition to the main block-wise Muon update, we considered a Dion2-inspired random column-partitioned variant as an exploratory ablation. The motivation was to introduce additional coupling between the two Monarch blocks of each factor while still avoiding Newton–Schulz orthogonalization of the full effective matrix.

For each Monarch factor, we concatenate its two trainable blocks along the column dimension according to the Monarch layer layout. For the L-factor, this gives

$$C_L = \mathrm{ColConcat}\Big(L^{(1)}, L^{(2)}\Big),$$

and for the corresponding momentum matrices,

$$M_L = \mathrm{ColConcat}\Big(M^{L^{(1)}}, M^{L^{(2)}}\Big).$$

At every periodic step, we randomly sample half of the columns of C_L. Let this random set of column indices be denoted by $\mathcal{I}_L$, and let $\overline{\mathcal{I}}_L$ be its complement. We then apply Newton–Schulz orthogonalization separately to the selected and unselected column groups:

$$O_L^{\mathcal{I}} = \mathrm{NS}(M_L[:, \mathcal{I}_L]), \qquad O_L^{\overline{\mathcal{I}}} = \mathrm{NS}\big(M_L[:, \overline{\mathcal{I}}_L]\big).$$

The two column groups are updated independently, scattered back into the concatenated matrix C_L, and then split back into $L^{(1)}$ and $L^{(2)}$ using the original Monarch block boundaries. The same random column-partitioned procedure is applied to the R-factor.

Here, $\mathrm{ColSplit}_{\mathrm{orig}}$ denotes splitting a concatenated matrix back according to the original Monarch block boundaries.

This strategy differs from the purely block-wise update because the random column split changes over training. As a result, different columns from the original Monarch blocks are periodically grouped together, allowing the optimizer to mix information across blocks over time. At the same time, each Newton–Schulz computation remains smaller than the full dense update. Thus, the method combines the structure of Monarch matrices [3] with ideas from recent Muon variants that shrink or periodically modify the orthogonalization step [6, 7].

However, as shown in Section 7, this additional random column coupling does not noticeably improve iteration-wise convergence over the simpler block-wise Monarch Muon update in our experiments. Therefore, we treat this method as an ablation rather than as the main proposed optimizer.

Algorithm 2 Random column-partitioned Muon update for Monarch layers

Require: Learning rate α, momentum coefficient β, period q
Require: Monarch blocks $L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}$ constructed by the Monarch layer
Initialize $M_0^B \leftarrow 0$ for all $B \in \{L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}\}$
for training step $t = 1, 2, \ldots$ **do**
  Compute loss $\mathcal{L}(\Theta_{t-1})$ and gradients by backpropagation
  for each block $B \in \{L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}\}$ **do**
    $G_t^B \leftarrow \nabla_B \mathcal{L}(\Theta_{t-1})$
    $M_t^B \leftarrow \beta M_{t-1}^B + G_t^B$
  end for
  if $t \bmod q = 0$ **then**
    for each factor $F \in \{L, R\}$ **do**
      $C_F \leftarrow \mathrm{ColConcat}(F_{t-1}^{(1)}, F_{t-1}^{(2)})$
      $M_F \leftarrow \mathrm{ColConcat}(M_t^{F^{(1)}}, M_t^{F^{(2)}})$
      Sample random $\mathcal{I}_F$ such that

$$|\mathcal{I}_F| = \frac{1}{2}\,\mathrm{cols}(C_F)$$

      Let $\overline{\mathcal{I}}_F$ be the complement of $\mathcal{I}_F$
      $C_F[:, \mathcal{I}_F] \leftarrow C_F[:, \mathcal{I}_F] - \alpha\,\mathrm{NS}_K(M_F[:, \mathcal{I}_F])$
      $C_F[:, \overline{\mathcal{I}}_F] \leftarrow C_F[:, \overline{\mathcal{I}}_F] - \alpha\,\mathrm{NS}_K(M_F[:, \overline{\mathcal{I}}_F])$
      $(F_t^{(1)}, F_t^{(2)}) \leftarrow \mathrm{ColSplit}_{\mathrm{orig}}(C_F)$
    end for
  else
    for each block $B \in \{L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}\}$ **do**
      $B_t \leftarrow B_{t-1} - \alpha\,\mathrm{NS}_K(M_t^B)$
    end for
  end if
  Reconstruct $\widetilde{W}_t = P L_t P^\top R_t$
end for

3.6 Properties of the proposed algorithms

The proposed algorithms have several useful properties. First, they reduce the size of the matrices used in Newton–Schulz orthogonalization. Standard Muon applies Newton–Schulz iterations to full dense matrix updates [1, 2]. In contrast, the proposed block-wise variant applies the same procedure to smaller Monarch factors. Since Newton–Schulz iterations rely on matrix multiplications, reducing matrix size directly reduces the computational cost of the orthogonalization step.

Second, the block-wise Newton–Schulz computations are independent after gradients have been computed. Therefore, the updates of $L^{(1)}$, $L^{(2)}$, $R^{(1)}$, and $R^{(2)}$ can be performed in parallel. This parallels the motivation behind distributed or parallel orthogonalized-update methods such as Dion [5], but the source of parallelism in our method comes from the structured parameterization rather than from a distributed reformulation of the optimizer.

Third, the random column-partitioned update was evaluated as an ablation to test whether additional random coupling between Monarch blocks improves convergence. In our experiments, this additional mechanism did not provide a noticeable improvement over the simpler block-wise update. This suggests that the main practical benefit comes from reducing the size of the matrices used in Newton–Schulz orthogonalization, rather than from random column repartitioning.

Finally, the proposed method changes the optimization problem. The model no longer optimizes over all dense matrices W, but over structured parameters $L^{(1)}$, $L^{(2)}$, $R^{(1)}$, and $R^{(2)}$. This can reduce computational cost, but it may also reduce expressivity. Therefore, the method introduces a trade-off between efficiency and model quality, similar to the general trade-offs studied in structured matrix parameterizations [3].

4 Related Work

4.1 Muon and orthogonalized optimizers

Muon was introduced as an optimizer that applies orthogonalized updates to matrix-valued parameters, with practical implementations relying on Newton–Schulz iterations to approximate the polar factor [1]. Recent work demonstrated that Muon can be scaled to large language model training when combined with suitable update scaling and weight decay choices [2]. This result motivates the study of Muon as a competitive alternative to standard adaptive optimizers such as AdamW, while also highlighting the importance of reducing the additional cost introduced by orthogonalization.

Several works improve or modify Muon-type updates. Scion studies training with norm-constrained linear minimization oracles and introduces normalization mechanisms that are related to the stability of orthogonalized updates [4]. Dion proposes distributed orthonormalized updates and replaces Newton–Schulz-style orthogonalization with a power-iteration-based approach that is better suited for parallel execution [5]. Dion2 reduces the size of the matrix used in Muon by applying orthogonalization to smaller randomly selected submatrices [6]. MuonBP proposes block-periodic orthogonalization, reducing the frequency or scope of expensive orthogonalization steps [7]. In contrast to these works, our method does not primarily modify Muon itself, but changes the structure of the trainable matrices to make the orthogonalization step cheaper.

4.2 Improving Newton–Schulz orthogonalization

Another line of work focuses directly on improving the Newton–Schulz iteration or related matrix sign methods. Gram Newton–Schulz proposes a modified formulation of the Newton–Schulz procedure for Muon-style optimization [8]. Polar Express studies optimal matrix sign methods and their application to the Muon algorithm [9]. These methods are complementary to our approach: they aim to make the orthogonalization procedure itself more efficient or more accurate, whereas we reduce the size of the matrices to which orthogonalization is applied.

4.3 Structured matrix parameterizations

Structured matrices provide a different way to reduce computational cost by restricting the class of trainable weight matrices. Monarch matrices represent a dense-like linear transformation as a product

of permutation and block-diagonal matrices and were shown to provide efficient and accurate training in neural networks [3]. Our work builds on this representation by using Monarch-parameterized layers as a mechanism for reducing the cost of Muon. Instead of applying Newton–Schulz iterations to a full dense matrix update, we apply them to smaller block-structured factors, which can reduce computational cost and expose additional parallelism.

5 Preliminaries

5.1 General notation

In this section, we introduce the general notation used in the rest of the paper. We denote matrices by uppercase letters, vectors by lowercase letters, and trainable model parameters by Θ. For a matrix A, $\|A\|_F$ denotes its Frobenius norm. The notation $\mathrm{NS}_K(A)$ denotes the result of applying K Newton–Schulz iterations to the matrix A. For a matrix C, the notation $C[:, \mathcal{I}]$ denotes the submatrix formed by selecting the columns indexed by $\mathcal{I}$.

5.2 Monarch layer structure

For each selected dense layer, we replace the original weight matrix with a Monarch-parameterized matrix

$$\widetilde{W} = PLP^\top R,$$

where P is a fixed permutation matrix, and L and R are trainable block-diagonal factors. In our implementation, each factor consists of two trainable blocks:

$$L = \mathrm{diag}(L^{(1)}, L^{(2)}), \qquad R = \mathrm{diag}(R^{(1)}, R^{(2)}).$$

The dimensions of these blocks are determined by the Monarch layer construction and by the input and output dimensions of the replaced dense layer. They are not additional assumptions about the data, the model distribution, or the optimal dense weight matrix. Rather, they are part of the structured architecture used in place of the dense layer.

For the random column-partitioned update, we use the block layout produced by the Monarch parameterization. The two blocks of each factor are concatenated along the column dimension according to this layout, updated, and then split back using the original block boundaries. Thus, the splitting operation is determined by the structured layer construction, not by an additional equal-size-block assumption.

6 Method

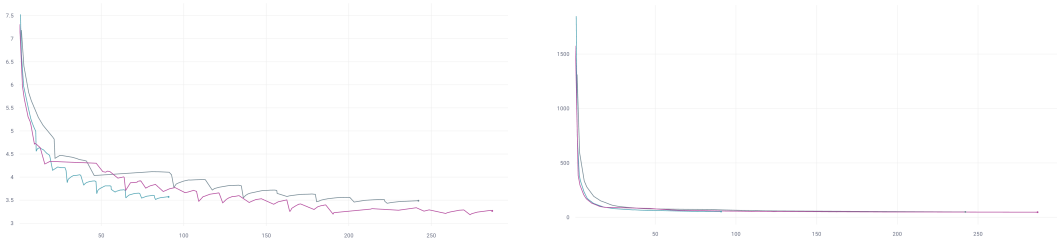
The proposed method replaces selected dense layers of a model with large matrix-valued parameters by Monarch-parameterized layers [3]. The resulting trainable parameters are the block factors $L^{(1)}$, $L^{(2)}$, $R^{(1)}$, and $R^{(2)}$. We then apply Muon-style orthogonalized updates to these factors rather than to the full effective matrix $\widetilde{W}$.

Our main method is the purely block-wise strategy, where each Monarch block is updated independently using Newton–Schulz orthogonalization. We also evaluate an additional Dion2-inspired random column-partitioned variant as an ablation. In this variant, the blocks of each Monarch factor are periodically concatenated according to the Monarch layout, randomly partitioned by columns, orthogonalized in two groups, and then split back using the original block boundaries. As shown in the experiments, this additional coupling does not provide a noticeable improvement over the simpler block-wise Monarch Muon update in our setting.

7 Experiments

To evaluate the proposed structured Muon variant, we instantiate the general problem from Section 2 on a compact GPT-2 style language model. The model has the following configuration:

$$n_{\mathrm{layer}} = 4, \qquad n_{\mathrm{head}} = 4, \qquad n_{\mathrm{embd}} = 384.$$



(a) Training loss as a function of optimization-step time.

(b) Validation perplexity as a function of optimization-step time.

Figure 1: Comparison of the considered optimizers under the same number of training epochs. The light blue curve corresponds to Monarch Muon, the gray curve corresponds to standard Muon, and the purple curve corresponds to AdamW. Although Monarch Muon converges slightly worse when measured by optimization iterations, it reaches comparable perplexity in less optimization-step time. This indicates that the reduction in Newton–Schulz orthogonalization cost can compensate for the reduced expressivity of the structured parameterization.

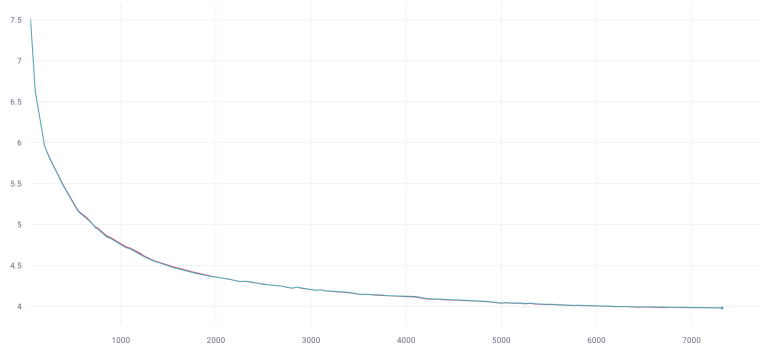


Figure 2: Training loss as a function of optimization iterations for block-wise Monarch Muon and the Dion2-inspired random column-partitioned Monarch Muon variant. The two curves are nearly identical, indicating that the additional random column coupling does not noticeably improve iteration-wise convergence in this experimental setting.

We compare AdamW, standard Muon, block-wise Monarch Muon, and a Dion2-inspired random column-partitioned Monarch Muon variant. The latter is included as an ablation rather than as the main proposed method. All methods were trained for the same number of epochs. Therefore, the comparison reflects not only convergence per iteration, but also the amount of wall-clock time spent on optimizer steps.

Following the motivation of Muon as a scalable optimizer for language model training [2], we use validation loss and perplexity as the main quality metrics. In addition, we explicitly track optimization-step time, since Newton–Schulz orthogonalization is the main additional cost of Muon-style methods [1, 8, 9].

The proposed method is considered successful if its final perplexity differs from the Muon or AdamW baseline by only a few percent while reducing the cost of orthogonalization through smaller and potentially parallel block-wise Newton–Schulz updates.

All algorithms were trained for the same number of epochs. This makes the comparison focused on the trade-off between optimization progress per iteration and optimization progress per unit of optimizer wall-clock time. Figure 1 shows that Monarch Muon may converge slightly worse when measured by iterations. This behavior is expected because the Monarch parameterization restricts the class of representable matrices compared with fully dense layers.

However, when training progress is measured as a function of optimization-step time, Monarch Muon becomes more favorable. Since Newton–Schulz iterations are applied to smaller block matrices, the optimizer step becomes cheaper, and the method reaches approximately the same validation

perplexity in less optimization time. In the plots, the light blue curve denotes Monarch Muon, the gray curve denotes standard Muon, and the purple curve denotes AdamW.

Figure 2 compares the basic block-wise Monarch Muon update with the Dion2-inspired random column-partitioned variant in terms of training loss versus optimization iterations. The curves are almost indistinguishable. This suggests that, in the considered compact GPT-2 setting, periodically mixing columns across Monarch blocks does not provide a meaningful convergence benefit over the simpler independent block-wise update.

Therefore, we use the block-wise Monarch Muon variant as the main structured optimizer in the remaining comparison. The Dion2-inspired variant remains useful as an ablation: it shows that the main gain of the proposed approach comes from reducing the size of the matrices used in Newton–Schulz orthogonalization, rather than from random column repartitioning.

This effect is expected to become more pronounced for larger models, where dense matrix orthogonalization is more expensive and accounts for a larger fraction of optimizer time. In the present work, we report experiments only for a compact GPT-2 style model because larger-scale experiments were not reproduced due to computational constraints. Nevertheless, the observed results support the main hypothesis that Monarch-parameterized layers can reduce the practical cost of Muon while preserving similar validation perplexity.

The proposed approach is complementary to recent optimizer-level improvements of Muon. Methods such as Dion, Dion2, MuonBP, Gram Newton–Schulz, and Polar Express modify the orthogonalization procedure, reduce the matrix size inside the optimizer, or improve the underlying matrix iteration [5–9]. In contrast, our method changes the parameterization of the model itself by introducing Monarch-structured layers [3]. This means that the computational gain comes from applying the same Muon principle to smaller structured factors.

At the same time, the method introduces a restriction on the hypothesis class of the model. A Monarch-parameterized layer cannot represent all dense matrices, so the main empirical question is whether the reduction in expressivity leads to a noticeable degradation in validation loss or perplexity. If the degradation remains small, the method can provide a practical way to reduce Muon's cost in settings where full dense orthogonalization is too expensive.

8 Conclusion

We studied a structure-based approach to reducing the computational cost of Muon. Instead of applying Muon to full dense layers, we replace selected layers with Monarch-parameterized matrices and apply Newton–Schulz orthogonalization to smaller trainable blocks. This reduces the cost of the optimizer step and creates opportunities for parallel block-wise computation.

We also considered a random column-partitioned update inspired by Dion2 and MuonBP, which periodically couples columns across the two blocks of each Monarch factor. However, this variant did not noticeably improve iteration-wise convergence over the simpler block-wise Monarch Muon update in our experiments. This suggests that the main benefit of the proposed approach comes from the structured Monarch parameterization and smaller block-wise Newton–Schulz computations, rather than from random column repartitioning.

Overall, our experiments indicate that Monarch-parameterized layers can reduce the practical cost of Muon while preserving similar validation perplexity in a compact GPT-2 style setting. We expect this effect to become more pronounced for larger models, where dense matrix orthogonalization accounts for a larger fraction of optimizer time. Larger-scale experiments remain an important direction for future work, but were not reproduced in this study due to computational constraints.

References

[1] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL `https://kellerjordan.github.io/posts/muon/`.

[2] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong

Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is scalable for llm training, 2025. URL https://arxiv.org/abs/2502.16982.

[3] Tri Dao, Beidi Chen, Nimit Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. Monarch: Expressive structured matrices for efficient and accurate training, 2022. URL https://arxiv.org/abs/2204.00595.

[4] Thomas Pethick, Wanyun Xie, Kimon Antonakopoulos, Zhenyu Zhu, Antonio Silveti-Falls, and Volkan Cevher. Training deep learning models with norm-constrained lmos, 2025. URL https://arxiv.org/abs/2502.07529.

[5] Kwangjun Ahn, Byron Xu, Natalie Abreu, Ying Fan, Gagik Magakyan, Pratyusha Sharma, Zheng Zhan, and John Langford. Dion: Distributed orthonormalized updates, 2025. URL https://arxiv.org/abs/2504.05295.

[6] Kwangjun Ahn, Noah Amsel, and John Langford. Dion2: A simple method to shrink matrix in muon, 2025. URL https://arxiv.org/abs/2512.16928.

[7] Ahmed Khaled, Kaan Ozkara, Tao Yu, Mingyi Hong, and Youngsuk Park. Muonbp: Faster muon via block-periodic orthogonalization, 2025. URL https://arxiv.org/abs/2510.16981.

[8] Jack Zhang, Noah Amsel, Berlin Chen, and Tri Dao. Gram newton-schulz, 2026. URL https://dao-ailab.github.io/blog/2026/gram-newton-schulz/.

[9] Noah Amsel, David Persson, Christopher Musco, and Robert M. Gower. The polar express: Optimal matrix sign methods and their application to the muon algorithm, 2026. URL https://arxiv.org/abs/2505.16932.

[10] Tom M Mitchell. The need for biases in learning generalizations. 1980.

[11] Jeremy Bernstein and Laker Newhouse. Old optimizer, new norm: An anthology, 2024. URL https://arxiv.org/abs/2409.20325.

[12] Jiaxiang Li and Mingyi Hong. A note on the convergence of muon, 2025. URL https://arxiv.org/abs/2502.02900.

[13] Alexey Kravatskiy, Ivan Kozyrev, Nikolai Kozlov, Alexander Vinogradov, Daniil Merkulov, and Ivan Oseledets. The ky fan norms and beyond: Dual norms and combinations for matrix optimization, 2025. URL https://arxiv.org/abs/2512.09678.

[14] Michael Crawshaw, Chirag Modi, Mingrui Liu, and Robert M. Gower. An exploration of non-euclidean gradient descent: Muon and its many variants, 2025. URL https://arxiv.org/abs/2510.09827.

[15] Kaja Gruntkowska, Alexander Gaponov, Zhirayr Tovmasyan, and Peter Richtárik. Error feedback for muon and friends, 2025. URL https://arxiv.org/abs/2510.00643.

[16] Aman Gupta, Rafael Celente, Abhishek Shivanna, D. T. Braithwaite, Gregory Dexter, Shao Tang, Hiroto Udagawa, Daniel Silva, Rohan Ramanath, and S. Sathiya Keerthi. Effective quantization of muon optimizer states, 2026. URL https://arxiv.org/abs/2509.23106.

[17] Dmitry Kovalev. Understanding gradient orthogonalization for deep learning via non-euclidean trust-region optimization, 2025. URL https://arxiv.org/abs/2503.12645.

A Appendix / supplemental material

A.1 Additional experiments / Proofs of Theorems

TODO